

WEEK 4 REPORT

FOOD ORDERING SYSTEM

Implementation of Orders and Order Details Module

Objective

The main objective of this week is to implement the Orders and Order Details tables for the Food Ordering System.

- Store customer order information.
- Store the food items included in each order.
- Maintain the quantity and price of ordered products.
- Connect orders with customers using a Foreign Key.
- Connect order details with products using a Foreign Key.
- Retrieve useful order information using JOIN queries.

Tables Used

CUSTOMER

Column	Data Type
CUSTOMER_ID	NUMBER(38)
CUSTOMER_NAME	VARCHAR2(100)
EMAIL	VARCHAR2(100)
PHONE	VARCHAR2(15)
ADDRESS	VARCHAR2(200)
REGISTRATION_DATE	DATE

The CUSTOMER table stores customer information.

ORDERS

Column	Data Type
ORDER_ID	VARCHAR2(10)
CUSTOMER_ID	NUMBER(38)
ORDER_DATE	DATE
TOTAL_AMOUNT	NUMBER(10,2)
ORDER_STATUS	VARCHAR2(20)

The ORDERS table stores information about customer orders.

ORDER_DETAILS

Column	Data Type
ORDER_DETAIL_ID	VARCHAR2(10)
ORDER_ID	VARCHAR2(10)
PRODUCT_ID	VARCHAR2(10)
QUANTITY	NUMBER
PRICE	NUMBER(10,2)

The ORDER_DETAILS table stores the individual food products included in an order.

Creation of ORDERS Table

```
SQL> CREATE TABLE ORDERS (Order_ID VARCHAR2(10), Customer_ID VARCHAR2(10), Order_Date DATE, Total_Amount NUMBER(10,2), Order_Status VARCHAR2(20));  
Table created.
```

Creation of ORDER_DETAILS Table

```
SQL> CREATE TABLE ORDER_DETAILS (Order_Detail_ID VARCHAR2(10), Order_ID VARCHAR2(10), Product_ID VARCHAR2(10), Quantity NUMBER, Price NUMBER(10,2));  
Table created.
```

Primary Key Constraints

The primary key uniquely identifies each record.

Orders

```
SQL> ALTER TABLE ORDERS ADD CONSTRAINT PK_Orders PRIMARY KEY (Order_ID);  
Table altered.
```

Order Details

```
SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT PK_OrderDetails PRIMARY KEY (Order_Detail_ID);  
Table altered.
```

Foreign Key Constraints

The CUSTOMER_ID in the ORDERS table is connected to the CUSTOMER table.

```
SQL> ALTER TABLE ORDERS ADD CONSTRAINT FK_Orders_Customer FOREIGN KEY (Customer_ID) REFERENCES CUSTOMER(Customer_ID);  
Table altered.
```

The PRODUCT_ID in the ORDER_DETAILS table is connected to the PRODUCT table.

```
SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT FK_OD_Product FOREIGN KEY (Product_ID) REFERENCES PRODUCT(Product_ID);  
Table altered.
```

These relationships maintain referential integrity between the tables.

Constraints Used

NOT NULL Constraint

The order date must contain a value.

```
SQL> ALTER TABLE ORDERS MODIFY Order_Date NOT NULL;  
Table altered.
```

Total Amount Check

The total amount cannot be negative.

```
SQL> ALTER TABLE ORDERS ADD CONSTRAINT CHK_Total CHECK (Total_Amount >= 0);  
Table altered.
```

Order Status Check

For the Food Ordering System:

```
SQL> ALTER TABLE ORDERS ADD CONSTRAINT CHK_Status CHECK (Order_Status IN ('Pending', 'Preparing', 'Delivered'));  
Table altered.
```

Quantity Check

The quantity must be greater than zero.

```
SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT CHK_Qty CHECK (Quantity > 0);  
Table altered.
```

Price Check

The product price cannot be negative.

```
SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT CHK_OD_Price CHECK (Price >= 0);  
Table altered.
```

Inserting Records into ORDERS

Five order records were inserted into the ORDERS table.

```
SQL> INSERT INTO ORDERS VALUES ('&ORDER_ID', &CUSTOMER_ID, TO_DATE('&ORDER_DATE', 'YYYY-MM-DD'), &TOTAL_AMOUNT, '&ORDER_STATUS');
```

```
SQL> /
```

```
Enter value for order_id: 106
```

```
Enter value for customer_id: 101
```

```
Enter value for order_date: 2026-09-23
```

```
Enter value for total_amount: 550
```

```
Enter value for order_status: Pending
```

```
old 1: INSERT INTO ORDERS VALUES ('&ORDER_ID', &CUSTOMER_ID, TO_DATE('&ORDER_DATE', 'YYYY-MM-DD'), &TOTAL_AMOUNT, '&ORDER_STATUS')
```

```
new 1: INSERT INTO ORDERS VALUES ('106', 101, TO_DATE('2026-09-23', 'YYYY-MM-DD'), 550, 'Pending')
```

```
1 row created.
```

```
SQL> /
```

```
Enter value for order_id: 107
```

```
Enter value for customer_id: 102
```

```
Enter value for order_date: 2026-05-15
```

```
Enter value for total_amount: 1250
```

```
Enter value for order_status: Delivered
```

```
old 1: INSERT INTO ORDERS VALUES ('&ORDER_ID', &CUSTOMER_ID, TO_DATE('&ORDER_DATE', 'YYYY-MM-DD'), &TOTAL_AMOUNT, '&ORDER_STATUS')
```

```
new 1: INSERT INTO ORDERS VALUES ('107', 102, TO_DATE('2026-05-15', 'YYYY-MM-DD'), 1250, 'Delivered')
```

```
1 row created.
```

```
SQL> /
```

```
Enter value for order_id: 108
```

```
Enter value for customer_id: 103
```

```
Enter value for order_date: 2026-03-09
```

```
Enter value for total_amount: 480
```

```
Enter value for order_status: Preparing
```

```
old 1: INSERT INTO ORDERS VALUES ('&ORDER_ID', &CUSTOMER_ID, TO_DATE('&ORDER_DATE', 'YYYY-MM-DD'), &TOTAL_AMOUNT, '&ORDER_STATUS')
```

```
new 1: INSERT INTO ORDERS VALUES ('108', 103, TO_DATE('2026-03-09', 'YYYY-MM-DD'), 480, 'Preparing')
```

```
1 row created.
```

```
SQL> /
```

```
Enter value for order_id: 109
```

```
Enter value for customer_id: 104
```

```
Enter value for order_date: 2026-05-10
```

```
Enter value for total_amount: 890
```

```
Enter value for order_status: Preparing
```

```
old 1: INSERT INTO ORDERS VALUES ('&ORDER_ID', &CUSTOMER_ID, TO_DATE('&ORDER_DATE', 'YYYY-MM-DD'), &TOTAL_AMOUNT, '&ORDER_STATUS')
```

```
new 1: INSERT INTO ORDERS VALUES ('109', 104, TO_DATE('2026-05-10', 'YYYY-MM-DD'), 890, 'Preparing')
```

```
1 row created.
```

```
SQL> /
```

```
Enter value for order_id: 110
```

```
Enter value for customer_id: 105
```

```
Enter value for order_date: 2026-10-10
```

```
Enter value for total_amount: 900
```

```
Enter value for order_status: Delivered
```

```
old 1: INSERT INTO ORDERS VALUES ('&ORDER_ID', &CUSTOMER_ID, TO_DATE('&ORDER_DATE', 'YYYY-MM-DD'), &TOTAL_AMOUNT, '&ORDER_STATUS')
```

```
new 1: INSERT INTO ORDERS VALUES ('110', 105, TO_DATE('2026-10-10', 'YYYY-MM-DD'), 900, 'Delivered')
```

```
1 row created.
```

Inserting Records into ORDER_DETAILS

Five order-detail records were inserted.

```
SQL> INSERT INTO ORDER_DETAILS VALUES ( '&ORDER_DETAIL_ID', '&ORDER_ID', '&PRODUCT_ID', &QUANTITY, &PRICE);
Enter value for order_detail_id: OD101
Enter value for order_id: 106
Enter value for product_id: P01
Enter value for quantity: 2
Enter value for price: 250
old 1: INSERT INTO ORDER_DETAILS VALUES ( '&ORDER_DETAIL_ID', '&ORDER_ID', '&PRODUCT_ID', &QUANTITY, &PRICE)
new 1: INSERT INTO ORDER_DETAILS VALUES ( 'OD101', '106', 'P01', 2, 250)

1 row created.
```

```
SQL> /
Enter value for order_detail_id: OD102
Enter value for order_id: 107
Enter value for product_id: P02
Enter value for quantity: 3
Enter value for price: 240
old 1: INSERT INTO ORDER_DETAILS VALUES ( '&ORDER_DETAIL_ID', '&ORDER_ID', '&PRODUCT_ID', &QUANTITY, &PRICE)
new 1: INSERT INTO ORDER_DETAILS VALUES ( 'OD102', '107', 'P02', 3, 240)

1 row created.
```

```
SQL> /
Enter value for order_detail_id: OD103
Enter value for order_id: 108
Enter value for product_id: P03
Enter value for quantity: 5
Enter value for price: 250
old 1: INSERT INTO ORDER_DETAILS VALUES ( '&ORDER_DETAIL_ID', '&ORDER_ID', '&PRODUCT_ID', &QUANTITY, &PRICE)
new 1: INSERT INTO ORDER_DETAILS VALUES ( 'OD103', '108', 'P03', 5, 250)

1 row created.
```

```
SQL> /
Enter value for order_detail_id: OD104
Enter value for order_id: 109
Enter value for product_id: P05
Enter value for quantity: 2
Enter value for price: 300
old 1: INSERT INTO ORDER_DETAILS VALUES ( '&ORDER_DETAIL_ID', '&ORDER_ID', '&PRODUCT_ID', &QUANTITY, &PRICE)
new 1: INSERT INTO ORDER_DETAILS VALUES ( 'OD104', '109', 'P05', 2, 300)

1 row created.
```

```

SQL> /
Enter value for order_detail_id: OD105
Enter value for order_id: 110
Enter value for product_id: P06
Enter value for quantity: 4
Enter value for price: 400
old 1: INSERT INTO ORDER_DETAILS VALUES ( '&ORDER_DETAIL_ID', '&ORDER_ID', '&PRODUCT_ID', &QUANTITY, &PRICE)
new 1: INSERT INTO ORDER_DETAILS VALUES ( 'OD105', '110', 'P06', 4, 400)

1 row created.

```

Displaying Order

This query displays all records from the ORDER table and helps verify the inserted records.

```
SQL> SELECT * FROM ORDERS;
```

ORDER_ID	CUSTOMER_ID	ORDER_DAT	TOTAL_AMOUNT	ORDER_STATUS
106	101	23-SEP-26	550	Pending
107	102	15-MAY-26	1250	Delivered
108	103	09-MAR-26	480	Preparing
109	104	10-MAY-26	890	Preparing
110	105	10-OCT-26	900	Delivered

Displaying Order

This query displays all records from the ORDER_DETAIL and helps verify the inserted records.

```
SQL> SELECT * FROM ORDER_DETAILS ;
```

ORDER_DETA	ORDER_ID	PRODUCT_ID	QUANTITY	PRICE
OD101	106	P01	2	250
OD102	107	P02	3	240
OD103	108	P03	5	250
OD104	109	P05	2	300
OD105	110	P06	4	400

JOIN Query – Customer Order Details

This query displays Customer ID, Customer Name, Order ID, Order Date, Product ID, Quantity, Price and Subtotal. The subtotal is calculated using $QUANTITY * PRICE$.

```
SQL> SELECT
2     C.CUSTOMER_ID,
3     C.CUSTOMER_NAME,
4     O.ORDER_ID,
5     O.ORDER_DATE,
6     OD.PRODUCT_ID,
7     OD.QUANTITY,
8     OD.PRICE,
9     OD.QUANTITY * OD.PRICE AS SUBTOTAL
10  FROM CUSTOMER C
11  JOIN ORDERS O
12    ON C.CUSTOMER_ID = O.CUSTOMER_ID
13  JOIN ORDER_DETAILS OD
14    ON O.ORDER_ID = OD.ORDER_ID
15  ORDER BY C.CUSTOMER_ID, O.ORDER_DATE;
```

CUSTOMER_ID	CUSTOMER_NAME	ORDER_ID	ORDER_DATE	PRODUCT_ID	QUANTITY	PRICE	SUBTOTAL
101	SNEHA	106	23-SEP-26	P01	2	250.00	500.00
102	VIJAY KUMAR	107	15-MAY-26	P02	3	240.00	720.00
103	ANITHA	108	09-MAR-26	P03	5	250.00	1250.00
104	SURESH	109	10-MAY-26	P05	2	300.00	600.00
105	KAMESH	110	10-OCT-26	P06	4	400.00	1600.00

Total Quantity of Each Product

This query calculates the total quantity ordered for each food product.

```
SQL> SELECT P.PRODUCT_NAME, SUM(O.QUANTITY) AS TOTAL_QUANTITY FROM PRODUCT P JOIN ORDER_DETAILS O ON P.PRODUCT_ID=O.PRODUCT_ID GROUP BY P.PRODUCT_NAME;
```

PRODUCT_NAME	TOTAL_QUANTITY
BROWNIE	2
CHICKEN FRIED RICE	5
CHICKEN BURGER	4
IDLI	2
BUTTER NAAN	3

Customer Order Summary

This query displays the customer name, order ID, order date and total order amount.

```
SQL> SELECT CUSTOMER_NAME,ORDER_ID,ORDER_DATE,TOTAL_AMOUNT FROM CUSTOMER JOIN ORDERS ON CUSTOMER.CUSTOMER_ID=ORDERS.CUSTOMER_ID;
```

CUSTOMER_NAME	ORDER_ID	ORDER_DATE	TOTAL_AMOUNT
-----	-----	-----	-----
SNEHA	106	23-SEP-26	550.00
VIJAY KUMAR	107	15-MAY-26	1250.00
ANITHA	108	09-MAR-26	480.00
SURESH	109	10-MAY-26	890.00
KAMESH	110	10-OCT-26	900.00

SQL Concepts Used

- CREATE TABLE
- PRIMARY KEY
- FOREIGN KEY
- NOT NULL constraint
- CHECK constraint
- INSERT
- COMMIT
- SELECT
- INNER JOIN
- SUM() function
- GROUP BY
- ORDER BY
- Arithmetic calculation
- DATE conversion using TO_DATE()

Result

The Orders and Order Details module was implemented for the Food Ordering System. The tables store customer orders and their corresponding food products. Primary keys and foreign keys establish relationships between the tables. Check constraints ensure that invalid quantities, prices, total amounts and order statuses are not accepted. JOIN queries are used to retrieve meaningful information about customers, orders and food products.